

✍ Exercise 1 – Producer Reliability and Retries

A team operates a Kafka producer pushing $\lambda = 100,000$ events per second to a topic with replication factor $r = 3$. Each broker has independent failure probability $p = 0.005$ during any single write window.

Part A. Acknowledgements.

- (a) Compute the loss probability for one record under `acks=1`.
- (b) Compute the loss probability under `acks=all` with `min.insync.replicas = 2`.
- (c) Compute the loss probability under `acks=all` with `min.insync.replicas = 3`.
- (d) For each of the three configurations above, compute the *expected number of lost records per second* at $\lambda = 100,000$ rec/s.

$$(a) P_{(loss)} = 0.005$$

$$(b) P_{(loss)} = 0.005^2 = 2.5 \times 10^{-5}$$

$$(c) P_{(loss)} = 0.005^3 = 1.25 \times 10^{-7}$$

(d)	loss prob	expected losses
<code>acks=1</code>	5×10^{-3}	500
<code>acks=all, min=2</code>	2.5×10^{-5}	2.5
<code>acks=all, min=3</code>	1.25×10^{-7}	0.0125

Part B. Retries. A transient send error occurs on each individual attempt with probability $q = 0.04$. Retries are independent and back-off is $B = 100$ ms; each attempt itself takes $L_0 = 10$ ms.

- (e) Compute the expected number of attempts $\mathbb{E}[T]$ to deliver a record.
- (f) Compute the expected end-to-end latency $\mathbb{E}[\text{latency}]$.
- (g) The producer is configured with `retries` = $R = 5$ (so 6 total attempts). Compute the residual loss probability after retries.
- (h) Combine Part A(b) and Part B(c): with `acks=all`, `min.insync.replicas` = 2, $p = 0.005$, and the retry policy of (g), what is the overall probability that a record is *ultimately* lost?

$$(e) \mathbb{E}[T] = \frac{1}{1-q} = \frac{1}{0.96} \approx 1.04$$

$$(f) \mathbb{E}[\text{latency}] = \mathbb{E}[T] \cdot L_0 + (\mathbb{E}[T] - 1) \cdot B \\ = 1.04 \times 10 + (1.04 - 1) \times 100 \approx 14.6 \text{ ms}$$

$$(g) \mathbb{P}(\text{still failing}) = 0.04^6 = 4.096 \times 10^{-9}$$

$$(h) \mathbb{P}(\text{total loss}) = q^6 + (1 - q^6) \cdot p^2 \approx 4.096 \times 10^{-9} + 2.5 \times 10^{-5} \approx 2.5 \times 10^{-5}$$

Exercise 2 – Consumer Groups, Lag, and Delivery Semantics

An IoT pipeline ingests sensor readings into a Kafka topic with $P = 12$ partitions. The producer rate is $\lambda = 60,000$ rec/s. Each consumer in the group can process up to $\mu = 8,000$ rec/s. The consumer group currently has $C = 6$ consumers.

Part A. Capacity and lag.

- (a) How many partitions does each consumer handle on average?
- (b) Compute the aggregate consumer throughput. Is the system stable?
- (c) A traffic spike doubles the producer rate to $\lambda' = 120,000$ rec/s for $T_{\text{spike}} = 30$ seconds. Compute the lag N_0 accumulated by the end of the spike.
- (d) After the spike, the producer rate returns to λ . Compute the time $T_{\text{catch-up}}$ needed to drain the backlog N_0 .

(a) Because $C \leq P$, each consumer receives P/C partitions

$$P/C = 12/6 = 2 \text{ partitions}$$

(b) The number of active consumers is

$$C_{\text{active}} = \min(C, P) = \min(6, 12) = 6$$

The aggregate throughput is

$$C_{\text{active}} \cdot \mu = 6 \times 8000 = 48000 \text{ rec/s}$$

$$\lambda = 60000 \text{ req/s} > C_{\text{active}} \cdot \mu \Rightarrow \frac{dN}{dt} = 60000 - 48000 = 12000 \text{ req/s}$$

the system is not stable.

(C) lag accumulated during the spike $C=6$

$$\frac{dN}{dt} = \lambda' - C_{\text{active}} \cdot \mu = 120000 - 48000 = 72000$$

$$N_0 = 72000 \times 30 = 2160000 \text{ records}$$

$$(d) T_{\text{catch-up}} = \frac{N_0}{C_{\text{active}} \cdot \mu - \lambda}$$

$$\because \lambda (60000) > \text{throughput} (48000) \therefore 48000 - 60000 = -12000$$

The system is permanently under-provisioned.

Part B. Scaling and rebalancing. The team considers adding more consumers to handle higher loads.

- (e) What is the maximum useful number of consumers in this group? Why?
- (f) With the maximum useful number of consumers, what is the maximum sustainable producer rate $\lambda_{\max}$?
- (g) The team needs to support $\lambda^* = 150,000$ rec/s *without* changing μ . What is the minimum number of partitions P^* they should reconfigure the topic with?

(e) each partition can be assigned to exactly one consumer in the group

Any consumers beyond $P=12$ will stay idle because

$$C_{\text{active}} = \min(C, P) = 12.$$

(f) $\lambda_{\max} = 12 \times 8000 = 96000$ rec/s

(g) $P^* \cdot \mu \geq \lambda^* \quad \therefore P^* \geq \frac{\lambda^*}{\mu} = \frac{150000}{8000} = 18.75$

$\therefore$ a minimum of 19 partitions is required.

Part C. Delivery semantics. The application uses at-least-once delivery (commit after processing). The probability that a consumer crashes between processing a record and committing the offset is $\alpha = 5 \times 10^{-4}$ per record.

- (h) Compute the expected number of times each record is processed.
- (i) At $\lambda = 60,000$ rec/s, how many duplicate processings occur per hour on average?
- (j) The team is considering switching to exactly-once semantics using transactional commits. Briefly justify in mathematical terms why this would be necessary if the consumer also writes each record to a downstream non-idempotent SQL database.

$$(h) \quad E[\text{processings}] = 1 + \alpha = 1 + 5 \times 10^{-4} = 1.0005$$

$$(i) \quad \lambda = 60000 \quad \alpha = 0.0005$$

$$\text{Total records per hour} = 60000 \times 3600 = 21600000$$

$$\text{Duplicates per hour} = 21600000 \times 0.0005 = 10800$$

(j) With at-least-once delivery, the expected processing count is $1 + \alpha > 1$ meaning α fraction of the records are processed multiple times.

A non-idempotent SQL database cannot inherently discard these duplicates, those α extra processings will result in corrupted application state.